P2P File Sharing: Midterm Progress Report

Group 19 | CS 4390, Section 502 | November 11, 2026

Team contributions

Member / email	Contribution
Isaac Chester ijc230001@utdallas.edu	Implemented tracker socket setup and IP/port assignment; create, update, and GET handlers; tracker-file parsing and entry updates; and server multithreading. Tested with Aditya.
Aditya Dogra axd230126@utdallas.edu	Researched and integrated OpenSSL MD5 functionality; implemented LIST; assisted with tracker creation; and tested with Isaac.
Maya Martinovic mtm230009@utdallas.edu	Implemented peer-side testing and command-line arguments for exercising the tracker methods, and handled client configuration files.
Conner Replogle clr220003@utdallas.edu	Helped create the Makefile, set up the GitHub repository, and designed the project folder structure.

Completed implementation and testing

Our development has focused on the tracker server, building on the instructor-provided starter code. The server supports **createtracker, updatetracker, LIST, and GET**. It creates tracker files, updates peer byte ranges and timestamps, lists available files, and returns tracker contents with an MD5 checksum. The current server creates a POSIX worker thread for each accepted connection; the worker handles the request and closes the socket. OpenSSL provides the MD5 implementation.

The peer supports command-line options for manually testing each tracker operation and reads client configuration files. We successfully tested communication between **a laptop at home acting as the peer and a PC acting as the tracker server**. The received tracker file was saved to disk. The accompanying screenshots document command tests and the thread implementation; the shown tracker entries use localhost test data. Full peer-to-peer shared-file transfers remain under development.

Remaining work

Next, we will add peer multithreading, simultaneous outgoing connections to multiple peers, peer-to-peer transfers for different file types, peer-side MD5 verification, and an interactive command-line file-selection interface. To complete the specification, we also plan sequential segment selection, newest-timestamp peer selection, nonoverlapping chunk downloads and assembly, enforcement of the 1,024-byte request limit, periodic tracker refreshes, recovery from unavailable peers, and resumption of incomplete downloads. End-to-end testing and the final demonstration starter script will follow.

Challenges and resources

Moving from C++ and Python to C required learning unfamiliar functions and memory-handling patterns. Changing between campus and home networks also changed our IP addresses; we learned to update network settings accordingly. Understanding the POSIX thread library was another learning curve that we addressed while implementing the server workers.

Resources: instructor-provided starter code and *Socket_Programming_Tutorial.pdf*; OpenSSL (openssl.org) for MD5; and GitHub Copilot to understand C functions and identify relevant APIs.

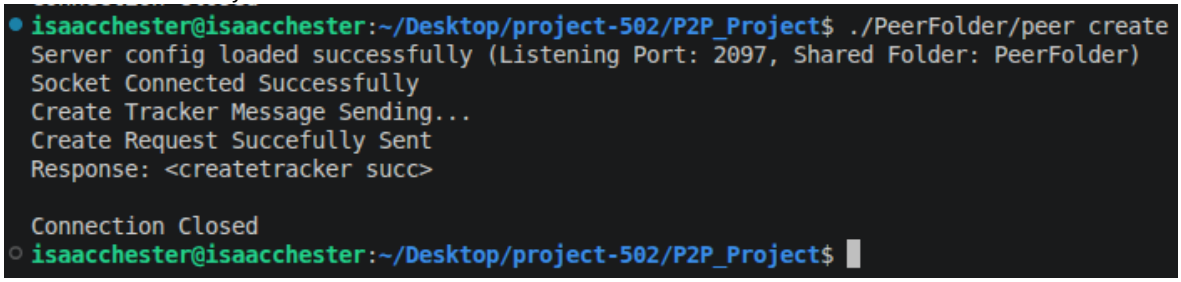


Figure 1. Peer create command receives a successful tracker-creation response.

Implementation and test evidence

```
isaacchester@isaacchester:~/Desktop/project-502/P2P_Project$ ./PeerFolder/peer update
Server config loaded successfully (Listening Port: 2097, Shared Folder: PeerFolder)
Socket Connected Successfully
Update Request sent Succefully
Tracker response: <updatetracker test_filename.avi succ>
Connection Closed
isaacchester@isaacchester:~/Desktop/project-502/P2P_Project$
```

Figure 2. Peer update command receives a successful response for test_filename.avi.

```
isaacchester@isaacchester:~/Desktop/project-502/P2P_Project$ ./PeerFolder/peer list
Server config loaded successfully (Listening Port: 2097, Shared Folder: PeerFolder)
Socket Connected Successfully
list request
List request written to server
<REP LIST 3>
<1 test_filename.avi 100400 00000000000000000000000000000000>
<2 test.txt 5000 9e107d9d372bb6826bd81d3542a419d6>
<3 test2.txt 60000 9e107d9d372bb6826bd81d3542a419dfs>
<REP LIST END>
Connection closed
isaacchester@isaacchester:~/Desktop/project-502/P2P_Project$
```

Figure 3. LIST returns three entries with names, sizes, and MD5 fields.

```
isaacchester@isaacchester:~/Desktop/project-502/P2P_Project$ ./PeerFolder/peer get
Server config loaded successfully (Listening Port: 2097, Shared Folder: PeerFolder)
Socket Connected Successfully
Get request was sent
<REP GET BEGIN>
Filename: test_filename.avi
Filesize: 100400
Description: Test_description
MD5: 00000000000000000000000000000000
127.0.0.1:2097:2848:4200:1794295316
<REP GET END 61a0dda3a37078c889d40d5edd952444>
Connection closed
isaacchester@isaacchester:~/Desktop/project-502/P2P_Project$
```

Figure 4. GET returns tracker metadata, the updated 2848-4200 byte range, and an ending MD5. The all-zero metadata MD5 is test data, not proof of shared-file integrity.

```c
// Each worker needs its own socket number, not the reused loop variable.
int *worker_socket = malloc(sizeof(*worker_socket));
if (worker_socket == NULL) {
    perror("Cannot allocate worker socket");
    close(sock_child);
    continue;
}
*worker_socket = sock_child;
pthread_t worker;
int error = pthread_create(&worker, NULL, peer_worker, worker_socket);
if (error != 0) {
    fprintf(stderr, "Cannot create worker: %s\n", strerror(error));
    free(worker_socket);
    close(sock_child);
}
```

Figure 5. Server allocates a socket argument per connection and calls pthread_create to start a worker, with error handling.